

Лабораторна робота №4

З курсу «Основи програмної інженерії»

Підготував студент ПЗПІ-25-3

Ільченко Віталій

Тема: Рефакторинг коду та проведення Code Review за індустріальними стандартами.

Мета: Набуття практичних навичок із проведення рецензування коду (Code Review) через GitHub Pull Requests; опанування методів ідентифікації та класифікації типових проблем якості коду (Code Smells); застосування рефакторинг-операцій для оптимізації внутрішньої структури ПЗ із збереженням його зовнішньої поведінки за допомогою регресійного тестування.

- Результати Code Review:

№	Рядок коду / Метод	Проблема (Дефект)	Категорія (Code Smell)	Рекомендаці я щодо виправлення
1	<code>def validate_file(self, filename, size_mb):</code>	Метод занадто великий і виконує декілька дій одночасно: шукає розширення через індекси рядка, перевіряє ліміти та валідує формат. Порушено принцип єдиної відповідальності (SRP).	Довгий метод	Застосувати рефакторинг <i>Extract Method</i> (Виділення методу). Створити допоміжний приватний метод <code>_get_extension</code> тільки для парсингу назви файлу.
2	<code>self.max_size_mb = 20 if size_mb > 10:</code>	Числові значення лімітів розміру файлу (20, 10) та фінансові коефіцієнти тарифів (2.0, 5.0)	Магічні числа	Застосувати операцію <i>Replace Magic Number with Constant</i> . Винести всі

	price += 2.0	жорстко прописані прямо всередині логіки методів. Це ускладнює налаштування програми.		чисельні константи на рівень класу з прозорими назвами (наприклад, MAX_SIZE_M B = 20).
3	Складна структура розрахунку вартості з декількома вкладеними блоками if/else.	Велика вкладеність умовних операторів робить код заплутаним і підвищує цикломатичну складність програми.	Надмірна Складність	Спростити умовні вирази за допомогою патерну <i>Guard Clauses</i> (раннє повернення результату через return), що дозволить вирівняти код лінійно.
4	<pre>for fmt in self.supported_formats: if extension == fmt:</pre>	Перевірка підтримуваних форматів реалізована через ітераційний цикл for по списку list. Це неефективно, оскільки пошук має лінійну складність.	Неефективна структура даних	Заміни тип колекції і списку list на хеш-таблицю set (множину). Це дозволить миттєво перевіряти наявність формату за

				допомогою оператора in зі швидкістю.
5	<pre># print("перевірка тексту..") # old_calc = size * 1.2</pre>	Усередині файлу присутні старі закоментовані шматки коду, відладочні принти та змінні, які більше ніде не викликаються в програмі.	Мертвий код	Повністю видалити всі закоментовані та неактивні рядки. Для збереження історії розробки та попередніх версій логіки має використовуватися система контролю версій Git.

- Посилання на Pull Request Code Review (URL).: <https://github.com/Vitaliillchenko/bse-lr1-ilchenko/compare/main%40%7B1day%7D...main>

- Результати рефакторингу:

№1) Було:

```
def validate_file(self, filename, size_mb):
```

```
    if size_mb <= 0:
```

```
        raise ValueError("Розмір файлу не може бути нульовим або
від'ємним")
```

```
# Ручний пошук розширення прямо в методі валідації
```

```
if '.' not in filename:
```

```
    return False, "Файл не має розширення"
```

```
extension = filename[filename.rfind('.').lower()]
```

```
if extension not in self.supported_formats:
    return False, f"Формат {extension} не підтримується"
```

Стало:

```
def _get_extension(self, filename):
    """Допоміжний приватний метод для виділення розширення (Extract
    Method)"""
    if '.' not in filename:
        return None
    return filename[filename.rfind('.').lower()]

def validate_file(self, filename, size_mb):
    if size_mb <= 0:
        raise ValueError("Розмір файлу не може бути нульовим або
        від'ємним")

    extension = self._get_extension(filename)
    if not extension or extension not in self.supported_formats:
        return False, f"Формат {extension} не підтримується або відсутнє
        розширення"
```

Чому:

Винесення парсингу в окремий ізольований приватний метод `_get_extension` спростило читання коду, зменшило його довжину всередині основного методу та дозволило повторно використовувати цю логіку в інших частинах класу за потреби.

№2) Було:

```
def validate_file(self, filename, size_mb):
    # ... логіка перевірки ...
    if size_mb > 20: # Що означає 20?
        return False, "Файл занадто великий"

def calculate_processing_fee(self, size_mb, is_priority=False):
    price = 5.0 # Що це за тариф?
    if size_mb > 10: # Що за ліміт 10?
        price += 2.0
```

```
if is_priority:
    price *= 1.5
return round(price, 2)
```

Стало:

```
class SmartDocAI:
    # Статичні константи класу замість "магічних чисел"
    MAX_SIZE_MB = 20
    LARGE_FILE_THRESHOLD_MB = 10
    BASE_PRICE = 5.0
    LARGE_FILE_SURCHARGE = 2.0
    PRIORITY_MULTIPLIER = 1.5

    def validate_file(self, filename, size_mb):
        # логіка перевірки
        if size_mb > self.MAX_SIZE_MB:
            return False, f"Файл занадто великий (макс. {self.MAX_SIZE_MB}МБ)"

    def calculate_processing_fee(self, size_mb, is_priority=False):
        price = self.BASE_PRICE
        if size_mb > self.LARGE_FILE_THRESHOLD_MB:
            price += self.LARGE_FILE_SURCHARGE
        if is_priority:
            price *= self.PRIORITY_MULTIPLIER
        return round(price, 2)
```

Чому:

Усунення запаху коду Magic Numbers. Числа 20, 10, 5.0, 2.0 та 1.5 були розкидані по різних методах програми без будь-яких контекстних назв. Якщо в майбутньому бізнес-вимоги зміняться (наприклад, базова ціна стане 6.0 або максимальний розмір збільшать до 50 МБ), розробнику довелося б шукати й міняти ці цифри вручну по всьому файлу, що створює ризик появи багів

№3) Було:

```
def calculate_processing_fee(self, size_mb, is_priority=False):
    # Велика вкладеність умов через конструкції if/else
```

```

if size_mb > 10:
    price = 7.0
    if is_priority:
        price = price * 1.5
    else:
        price = price
else:
    price = 5.0
    if is_priority:
        price = price * 1.5
return round(price, 2)

```

Стало:

```

def calculate_processing_fee(self, size_mb, is_priority=False):
    """Розрахунок вартості (Рефакторинг: лінійна структура через Guard
    Clauses)"""
    price = self.BASE_PRICE

    # Покрокове лінійне нарахування без вкладеності
    if size_mb > self.LARGE_FILE_THRESHOLD_MB:
        price += self.LARGE_FILE_SURCHARGE

    if is_priority:
        price *= self.PRIORITY_MULTIPLIER

    return round(price, 2)

```

Чому:

Початкова реалізація містила заплутане дублювання розрахунку пріоритету всередині обох гілок if та else, створюючи "стрілоподібну" структуру відступів. Застосування лінійних перевірок дозволило позбутися блоків else, прибрати дублювання математичних операцій та зробити логіку розрахунку ціни очевидною та простою для тестування.

- Звіт регресійного тестування:

```

48     """BVA: 20.1МБ (За межею)"""
49     # Act
50     result, msg = self.ai.validate_file("big.pdf", 20.1)
51     # Assert

```

PROBLEMS OUTPUT **TERMINAL** PORTS ... powershell + v [] [X] [] [X]

```

PS C:\Users\Hobidon\bse-lr1-ilchenko> python -m unittest test_smartdoc.py
.....
-----
Ran 10 tests in 0.003s

OK
PS C:\Users\Hobidon\bse-lr1-ilchenko>

```

Ln 38, Col 50 Spaces: 4 UTF-8 LF Python 3.14.4 Python 3.14.4

- Порівняння метрик ДО/ПІСЛЯ:

Метрика якості програми	Значення ДО рефакторингу	Значення ПІСЛЯ рефакторингу	Архітектурне значення та ефект зміни
Цикломатична складність (Radon CC)	6	2	Завдяки застосуванню Guard Clauses (раннього повернення результату через return), розплетано вкладені блоки if/else.
Кількість зауважень лінтера (Ruff)	5 попереджень	0 попереджень	Повністю усунуено дефекти типу Magic Numbers (всі числа замінено на іменовані константи класу) та видалено Dead Code
Зауваження статичного аналізу (SonarLint)	3 issues	0 issues	Статичний аналізатор більше не фіксує порушення архітектурного принципу єдиної відповідальності, оскільки метод

			валідації файлу було декомпоновано.
Статус модульних тестів (unittest)	10/10 (успішно)	10/10 (успішно)	Показник залишився незмінним. Це головний критерій успішного рефакторингу: внутрішня структура коду покращилась, але його зовнішня поведінка не порушилась.

- Підсумкова рефлексія:

1. Проведення взаємного Code Review дозволило мені поглянути на власний код очима іншого розробника та навчитися помічати приховані дефекти архітектури, такі як «магічні числа» чи надмірна складність умов.
2. Я переконався, що своєчасне усунення цих «запахів коду» через рефакторинг робить програму значно зрозумілішою для подальшого супроводу та розширення.
3. Практичне поєднання рефакторингу з регресійним тестуванням дало впевненість у тому, що структурні зміни не порушили логіку роботи програми.
4. Чистота коду та автоматизована перевірка якості є невід'ємними складовими професійної розробки програмного забезпечення.